

SKY PLAY ENTERTAINMENT

DOSSIER DE CONCEPTION TECHNIQUE — PWA COMPAGNON

Architecture : Next.js (App Router) & SQLite | Charte Graphique : Gaming Cyan & Dark Navy

1. INTRODUCTION & OBJECTIFS DE LA PWA

Ce document détaille les spécifications techniques de la Progressive Web App (PWA) conçue comme outil compagnon pour la phase de test utilisateur de la plateforme **skyplay.cloud**. L'objectif principal est de fournir un environnement léger, indépendant et disponible hors ligne, permettant de simuler et de valider les quatre jalons du protocole de test sans nécessiter d'intégrations API complexes ou d'accès administrateur directs sur le cœur de l'application en cours de développement.

2. ARCHITECTURE TECHNIQUE & CHOIX TECHNOLOGIQUES

- **Framework** : Next.js 14+ (App Router) pour une structure de routes performante et l'optimisation SSR/ISR du côté applicatif.
- **Gestion PWA** : [@ducanh2912/next-pwa](#) pour la génération automatique des Service Workers et la gestion avancée du cache.
- **Base de Données** : SQLite (via le driver Node-sqlite3), permettant une base locale légère, intégrée directement dans le répertoire du projet sous forme de fichier binaire, idéale pour une phase de test agile.

3. MODÉLISATION DES DONNÉES (SCHÉMA SQLITE)

La base de données repose sur un modèle relationnel à trois entités principales permettant d'enregistrer l'identité des testeurs, de définir les jalons et de capturer les feedbacks accompagnés de leurs preuves visuelles (images encodées en chaînes Base64).

```

-- Table des Testeurs / Utilisateurs
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table des Jalons du Protocole
CREATE TABLE steps (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  slug VARCHAR(20) UNIQUE NOT NULL, -- 'jalon_1', 'jalon_2'...
  title VARCHAR(100) NOT NULL,
  reward_amount INTEGER NOT NULL
);

-- Table des Soumissions de Feedback et Captures d'Écran
CREATE TABLE submissions (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  step_id INTEGER NOT NULL,
  ux_feedback TEXT NOT NULL,
  screenshot_base64 TEXT NOT NULL,
  status VARCHAR(20) DEFAULT 'PENDING', -- PENDING, APPROVED, REJECTED
  submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (step_id) REFERENCES steps(id),
  UNIQUE(user_id, step_id)
);

```

4. SPÉCIFICATIONS D'INTERFACE & CHARTE GRAPHIQUE CSS

Afin de garantir une continuité visuelle parfaite avec l'arène de jeu officielle de **skyplay.cloud**, l'interface adopte une esthétique sombre résolument "gaming". Les couleurs clés extraites du support marketing incluent le bleu nuit abyssal, les accents cyan néon et les dégradés magenta/violet.

```

:root {
  --skyplay-bg-dark: #070f1e;      /* Fond principal profond */
  --skyplay-bg-card: #0d1b2e;      /* Arrière-plan des blocs de formulaires */
  --skyplay-cyan: #31e7df;         /* Cyan néon pour éléments interactifs */
  --skyplay-green: #2ecc71;        /* Vert pour succès et validations */
  --skyplay-orange: #e67e22;       /* Orange pour avertissements / attente */
  --skyplay-text-main: #ffffff;     /* Texte principal */
  --skyplay-text-muted: #8fa0ba;   /* Labels et textes secondaires */

  --gradient-button: linear-gradient(90deg, #31e7df 0%, #2ecc71 100%);
  --gradient-logo: linear-gradient(135deg, #00d2ff 0%, #9b5de5 50%, #f15bb5 100%);
}

body {
  background-color: var(--skyplay-bg-dark);
  color: var(--skyplay-text-main);
  font-family: 'Inter', system-ui, sans-serif;
}

.btn-skyplay-capsule {
  background: var(--gradient-button);
  color: #070f1e;
  font-weight: 700;
  text-transform: uppercase;
  padding: 12px 30px;
  border-radius: 50px;
  border: none;
  box-shadow: 0 0 15px rgba(49, 231, 223, 0.4);
  cursor: pointer;
}

```

5. WORKFLOW DU TRAITEMENT DES DONNÉES (API ROUTE)

Le serveur Next.js réceptionne le payload JSON contenant les chaînes textuelles du formulaire ainsi que la capture d'écran encodée en Base64. Après validation de l'intégrité des données, l'insertion se fait de manière atomique dans SQLite avec une contrainte d'unicité sur le couple (**user_id**, **step_id**) pour empêcher le double-déclaratif frauduleux.

Route API	Méthode	Payload Reçu	Action Système
/api/submit	POST	{ userId, stepId, uxFeedback, screenshot }	Vérifie l'unicité, insère la soumission en statut PENDING dans la table SQLite.
/api/admin/submissions	GET	Aucun (Headers d'auth requis)	Récupère la liste des feedbacks et les images Base64 pour modération visuelle.
/api/admin/approve	POST	{ submissionId, status }	Met à jour le statut (APPROVED/REJECTED) et incrémente virtuellement la cagnotte du testeur.

6. CONFIGURATION PWA & MANIFESTE

Pour permettre le comportement "standalone" (installation sur l'écran d'accueil sans barres de navigation du navigateur mobile), le fichier **manifest.json** définit les icônes adaptatives et fige l'orientation de l'application compagnon en mode portrait pour optimiser la saisie des feedbacks et le téléversement des captures d'écran.